A Course Based Project Report on

KEY-VALUE STORAGE SYSTEM

Submitted to the

Department of CSE-(CyS, DS) and AI&DS

in partial fulfilment of the requirements for the completion of course

WEB TECHNOLOGIES LABORATORY (22PC2CS302)

BACHELOR OF TECHNOLOGY

IN

CSE-Data Science

Submitted by

J. JAGADEESWAR	23071A67F4
K. PARVEZ ALAM	23071A67F5
K. VARSHITH REDDY	23071A67F7
M. KOUSTHUB	23071A67F8
M. NEHAL REDDY	23071A67G1

Under the guidance of

Mr. Mohd Ashfakul Hasan

Assistant Professor

TAMASOMA JYOTIRGAMAYA

Department of CSE- (CyS, DS) and AI&DS

VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF ENGINEERING & TECHNOLOGY

An Autonomous Institute, NAAC Accredited with 'A++' Grade, NBA

VignanaJyothi Nagar, Pragathi Nagar, Nizampet (S.O), Hyderabad – 500 090, TS, India

May - 2026

VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF ENGINEERING AND TECHNOLOGY

An Autonomous, ISO 21001:2018& QS I-Gauge Diamond Rated Institute, Accredited by NAAC with 'A++' Grade
NBA Accreditation for B.Tech. CE,EEE,ME,ECE,CSE,EIE,IT,AME, M.Tech. STRE, PE, AMS, SWEProgrammes
Approved by AICTE, New Delhi, Affiliated to JNTUH, NIRF (2024) Rank band:151-200in EngineeringCategory
College with Potential for Excellence by UGC,JNTUH-Recognized Research Centres:CE,EEE,ME,ECE,CSEVignana Jyothi Nagar, Pragathi Nagar, Nizampet (S.O.), Hyderabad – 500 090, TS, India.
Telephone No: 040-2304 2758/59/60, Fax: 040-23042761
E-mail: postbox@vnrvjiet.ac.in, Website: www.vnrvjiet.ac.in

Department of CSE-(CyS, DS) and AI&DS

CERTIFICATE

This is to certify that the project report entitled "**KEY-VALUE STORAGE SYSTEM**" is a bonafide work done under our supervision and is being submitted by **Mr. JAGADEESWAR (23071A67F4), Mr. K. PARVEZ ALAM (23071A67F5), Mr. K.VARSHITH REDDY (23071A67F7), Mr. M. KOUSTHUB (23071A67F8), Mr. M. NEHAL REDDY (23071A67G1)** in partial fulfillment for the award of the degree of **Bachelor of Technology** in **CSE-Data Science**, of the VNRVJIET, Hyderabad during the academic year 2025-2026.

Mr. Mohd Ashfakul Hasan
Assistant Professor
Dept of CSE-(CyS, DS) and AI&DS

Dr. G. Ramesh Chandra
Professor& I/C HOD
Dept of CSE-(CyS, DS) and AI&DS

Course based Projects Reviewer

VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institute, NAAC Accredited with 'A++' Grade,

VignanaJyothi Nagar, Pragathi Nagar, Nizampet(SO), Hyderabad-500090, TS, India

Department of CSE-(CyS, DS) and AI&DS

DECLARATION

We declare that the course based project work entitled "**KEY-VALUE STORAGE SYSTEM**" submitted in the Department of **CSE-(CyS, DS) and AI&DS**, Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering and Technology, Hyderabad, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in CSE-Data Science** is a bonafide record of our own work carried out under the supervision of **Mr. Mohd Ashfakul Hasan, Assistant Professor, Department of CSE-(CyS, DS) and AI&DS, VNRVJIET.** Also, we declare that the matter embodied in this thesis has not been submitted by us in full or in any part thereof for the award of any degree/diploma of any other institution or university previously.

Place: Hyderabad.

J. Jagadeeswar	**K. Parvez Alam**	**K. Varshith Reddy**	**M. Kousthub**	**M. Nehal Reddy**
(23071A67F4)	(23071A67F5)	(23071A67F7)	(23071A67F8)	(23071A67G1)

ACKNOWLEDGEMENT

We express our deep sense of gratitude to our beloved President, Sri. D. Suresh Babu, VNR Vignana Jyothi Institute of Engineering & Technology for the valuable guidance and for permitting us to carry out this project.

With immense pleasure, we record our deep sense of gratitude to our beloved Principal, **Dr. B Chennakesava Rao**, for permitting us to carry out this project.

We express our deep sense of gratitude to our beloved Professor **Dr.G.Ramesh Chandra**, Professor and I/C HOD, Department of CSE-(CyS, DS) and AI&DS, VNR Vignana Jyothi Institute of Engineering & Technology, Hyderabad-500090 for the valuable guidance and suggestions, keen interest and through encouragement extended throughout the period of project work.

We take immense pleasure to express our deep sense of gratitude to our beloved Guide, **Mr. Mohd Ashfakul Hasan**, Assistant Professor in CSE- (CyS, DS) and AI&DS, VNR Vignana Jyothi Institute of Engineering & Technology, Hyderabad, for his/her valuable suggestions and rare insights, for constant source of encouragement and inspiration throughout my project work.

We express our thanks to all those who contributed for the successful completion of our project work.

Mr. JAGADEESWAR (23071A67F4)

Mr. K. PARVEZ ALAM (23071A67F5)

Mr. K. VARSHITH REDDY (23071A67F7)

M. KOUSTHUB (23071A67F8)

M. NEHAL REDDY (23071A67G1)

TABLE OF CONTENTS

S No.	Title	Page No.
1	Abstract	2
1	Introduction	3
2	Methodology	4
3	Test Cases	9
4	Results	10
5	Conclusion	12
6	References	14

ABSTRACT

This project presents the design and implementation of a lightweight web-based key-value store that supports basic CRUD operations using HTTP methods such as GET, PUT, and DELETE. Developed using Java Servlets on Apache Tomcat, the system leverages an in-memory data structure, specifically Concurrent HashMap, to achieve high-speed and thread-safe data access in a multi-threaded environment. A simple JSP-based frontend provides an intuitive interface for user interaction, while the backend handles request routing, processing, and response generation. Unlike traditional database-driven systems, this approach eliminates external dependencies, reducing complexity and improving performance. The architecture emphasizes core backend principles such as RESTful API design, concurrency management, and efficient request handling. Additionally, an optional persistence mechanism can be integrated to store data across server restarts. This project serves as a practical demonstration of building scalable and efficient backend systems using fundamental Java web technologies

CHAPTER-1
INTRODUCTION

The rapid growth of web applications has increased the need for efficient and scalable data storage mechanisms. One of the simplest and most widely used data models is the key-value store, where data is stored as pairs of keys and corresponding values. This model forms the foundation of many high-performance systems such as caching layers and in-memory databases.

This project focuses on the design and development of a lightweight web-based key-value store using core Java technologies. The system is implemented using Java Servlets deployed on Apache Tomcat, providing a RESTful interface to perform basic operations such as storing, retrieving, and deleting data. Unlike traditional database-backed systems, the application uses an in-memory data structure, specifically ConcurrentHashMap, to ensure fast access and thread safety in a concurrent environment.

A simple JSP-based frontend is integrated to allow user interaction through a web browser, making the system easy to use and test. The project emphasizes understanding of backend architecture, HTTP request handling, and concurrent programming. By avoiding complex frameworks and external dependencies, it provides a clear view of how web servers, servlets, and data handling mechanisms work together in a real-world application

CHAPTER-2
METHODOLOGY

2. 1 Overview and Architecture

The system is designed as a lightweight, layered web application that processes client requests through a clear flow of components. At a high level, the architecture follows a client–server model where a browser-based frontend communicates with a backend deployed on Apache Tomcat using HTTP.

The frontend is implemented using a JSP page combined with JavaScript (Fetch API), which allows users to send asynchronous HTTP requests (GET, PUT, DELETE). These requests are received by the Tomcat server, which acts as a servlet container responsible for routing incoming requests based on configuration defined in web.xml.

The core processing is handled by a Servlet (StoreServlet), which serves as the controller layer. It interprets HTTP methods, extracts keys and values from the request, and delegates operations to the backend storage component. The storage layer is implemented using a singleton class (KVStore) that internally uses a ConcurrentHashMap. This ensures thread-safe access to shared data when multiple requests are processed simultaneously by Tomcat threads.

The architecture eliminates the need for external databases and ORM frameworks, focusing instead on in-memory data storage for high performance and reduced complexity. Optionally, persistence can be added using a lifecycle listener that writes data to a file during shutdown and reloads it during startup. This design demonstrates core backend principles such as request routing, concurrency handling, and RESTful API interaction .

2. Implementation Steps

The implementation of the system is carried out in a sequence of well-defined steps:

Step 1:

Environment Setup

The development environment is prepared by installing Java Development Kit (JDK), Eclipse IDE for Enterprise Java, and Apache Tomcat 9. Eclipse is configured to integrate with Tomcat, enabling direct deployment and execution of the web application.

Step 2:

Project Creation

A Dynamic Web Project named kv-store is created in Eclipse. The project structure is organized into source folders (src) for Java classes and WebContent for frontend resources. The WEB-INF directory is used to store configuration files such as web.xml.

Step 3:

Core Storage Implementation

A singleton class KVStore is implemented to manage data storage. It uses ConcurrentHashMap to store key-value pairs, providing efficient O(1) operations for insertion, retrieval, and deletion. The singleton pattern ensures a single shared instance across all requests.

Step 4: Servlet Development

The StoreServlet class is created by extending HttpServlet. It overrides methods such as doGet, doPut, and doDelete to handle corresponding HTTP requests. The servlet extracts the key from the URL and value from the request body, then calls appropriate methods in the KVStore.

Step 5: Request Routing Configuration

The web.xml file is configured to map the URL pattern /store/* to the StoreServlet. This enables dynamic routing of requests such as /store/name to the servlet for processing.

Step 6: Frontend Development

A JSP file (index.jsp) is developed to provide a user interface. It includes input fields for key and value, along with buttons to perform operations. JavaScript functions use the Fetch API to send asynchronous HTTP requests to the backend and display responses.

Step 7: Deployment and Execution

The application is deployed on Tomcat using Eclipse's "Run on Server" feature. The server compiles the project, deploys it, and hosts it on http://localhost:8080/kv-store/.

Step 8: Testing and Validation

The system is tested by performing CRUD operations through the web interface and using tools like curl or browser requests. The correctness of operations, response handling, and concurrency behavior is verified. Optional persistence can be tested by restarting the server and checking data recovery.

This methodology ensures a clear progression from setup to deployment while emphasizing core backend development concepts and system design principles.

IMPLEMENTATION

1. Data Storage Layer (KVStore.java)

```
private final ConcurrentHashMap<String, String> map = new ConcurrentHashMap<>();

public void put(String key, String value) { map.put(key, value); }
public String get(String key)             { return map.get(key); }
public boolean delete(String key)         { return map.remove(key) != null; }
```

The storage layer is implemented using ConcurrentHashMap, a thread-safe data structure provided by Java. It enables concurrent read and write operations without external synchronization, ensuring data integrity in a multi-threaded environment such as a web server. The use of bucket-level locking allows high performance under concurrent access. The class follows the Singleton design pattern, ensuring a single shared instance across the entire application lifecycle. This eliminates the need for external databases while maintaining efficient in-memory storage.

2. URL Routing Configuration (web.xml)

```
<servlet-mapping>
    <servlet-name>StoreServlet</servlet-name>
    <url-pattern>/store/*</url-pattern>
</servlet-mapping>
```

The web.xml file acts as the deployment descriptor, defining how incoming HTTP requests are mapped to specific servlet classes. The URL pattern /store/* routes all matching requests to the StoreServlet. The wildcard (*) captures dynamic path segments, which are later interpreted as keys. This centralized configuration enables clear and maintainable request routing without relying on annotations or external frameworks.

3. API Usage Examples

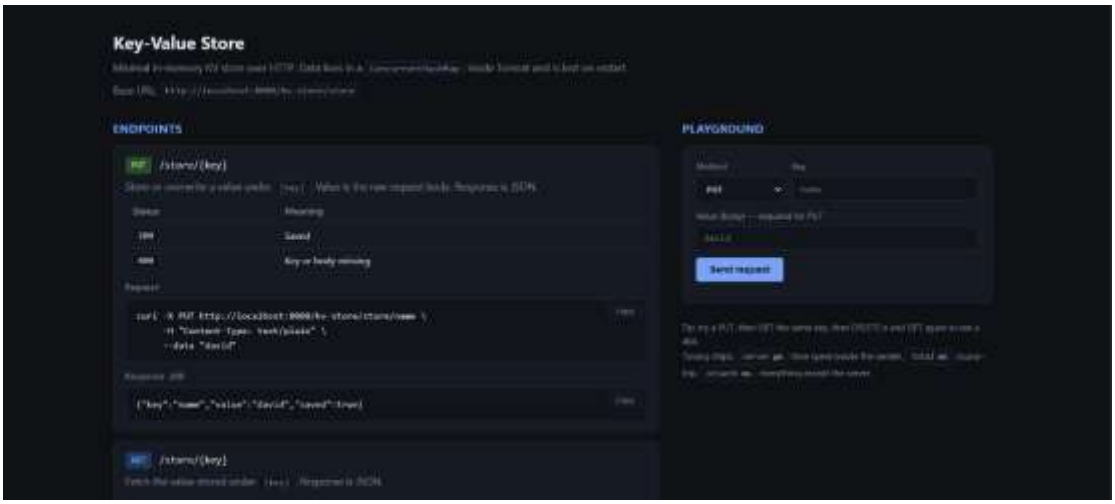
```
# Store a value
curl -X PUT -H "Content-Type: text/plain" \
     -d "Alice" http://<host>/store/name

# Retrieve the value
curl http://<host>/store/name

# Delete the value
curl -X DELETE http://<host>/store/name
```

The system follows RESTful API conventions where the key is embedded in the URL path and the value is transmitted in the request body for PUT operations. Responses are returned in a structured format, and appropriate HTTP status codes are used to indicate success or failure. For example, a missing key result in a 404 Not Found response. This design ensures simplicity, flexibility, and ease of integration with various clients.

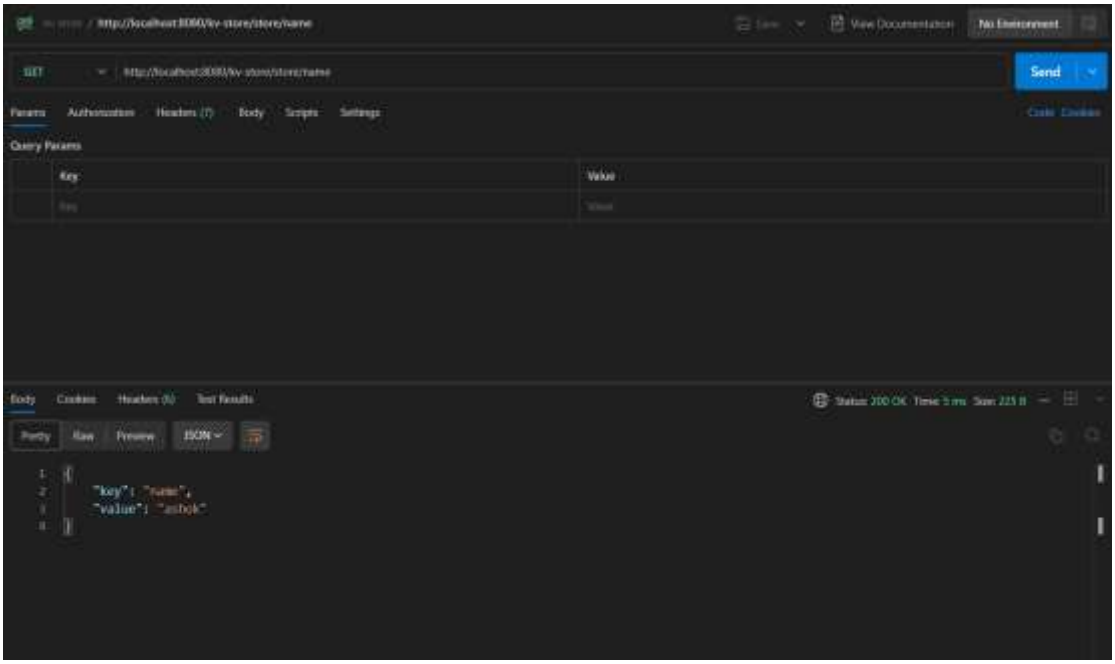
CHAPTER-3

TEST CASES/ OUTPUT

1. Web UI – API Documentation



2. Accessing from API Client



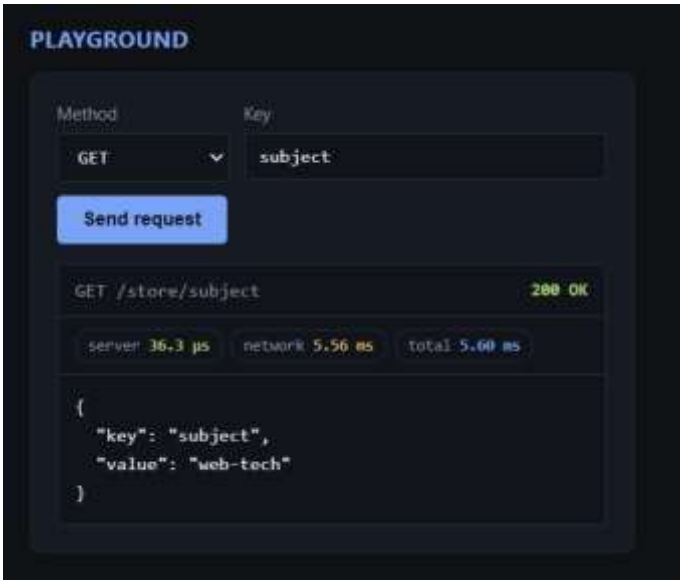
CHAPTER-4

RESULTS

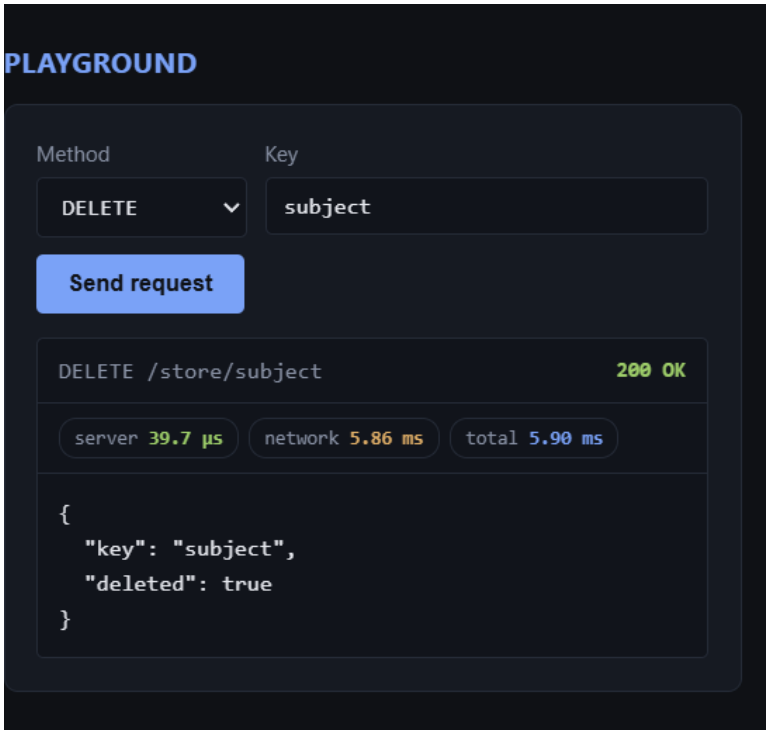
The implemented system successfully delivers a lightweight, in-memory key-value store accessible over HTTP using standard methods (GET, PUT, DELETE). All endpoints function correctly when tested both through the browser-based UI and via command-line tools such as curl. Example interactions confirm expected behavior:

1. PUT

2. GET



3. DELETE



Latency measurements show that the internal processing time remains extremely low (typically in the range of a few microseconds after JVM warm-up), while total request time is dominated by network and HTTP overhead (milliseconds). This validates the design choice of using an in-memory data structure, where computation is negligible compared to I/O.

The system is inherently thread-safe due to the use of ConcurrentHashMap, which allows multiple threads (spawned by Tomcat's request handling pool) to perform read and write operations concurrently without data corruption. No explicit synchronization is required, and the singleton pattern ensures a consistent shared state across all requests.

CHAPTER 5

SUMMARY

The project demonstrates the design and implementation of a lightweight, web-based key-value store using core Java web technologies. It follows a minimal architecture where a JSP-based frontend interacts with a backend built using Java Servlets deployed on Apache Tomcat. The system processes HTTP requests (GET, PUT, DELETE) and performs operations on an in-memory data structure using ConcurrentHashMap. This approach eliminates the need for external databases and complex frameworks, focusing instead on performance, simplicity, and clarity of backend operations. The project effectively illustrates key concepts such as RESTful API design, request handling, concurrency management, and server-side processing.

CONCLUSION

The developed system successfully achieves its objective of creating a simple and efficient key-value storage service accessible through a web interface. By leveraging in-memory storage and thread-safe data structures, the application ensures fast data access and reliable performance in a concurrent environment. The use of Java Servlets and Apache Tomcat provides a clear understanding of how web servers handle client requests and responses. Although the system sacrifices persistence in its basic form, it highlights the trade-offs between performance and durability. Overall, the project serves as a strong foundation for understanding backend development and system design principles.

RECOMMENDATIONS

To enhance the functionality and scalability of the system, several improvements can be considered. First, integrating a persistence layer using a database or file-based storage would allow data to survive server restarts. Second, implementing features such as key expiration (TTL), authentication mechanisms, and input validation would improve robustness and security. Third, extending the API to support JSON-based communication and additional endpoints (e.g., listing all keys or system statistics) would make the system more practical. Finally, migrating the project to a modern framework such as Spring Boot can help in building scalable, production-ready applications while maintaining clean architecture and maintainability.

REFERENCES

[1]. Apache Tomcat. Available at: https://tomcat.apache.org/ : Official Website

[2]. Apache Tomcat Overview. Available at:

https://en.wikipedia.org/wiki/Apache_Tomcat : Encyclopedia Article

[3]. Jakarta Servlet Specification. Available at:

https://jakarta.ee/specifications/servlet/ : Technical Specification

[4]. Oracle Java Documentation — ConcurrentHashMap. Available at:

https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/ConcurrentHashMap.html : Official API Documentation

[5]. Java ConcurrentMap Overview. Available at:

https://en.wikipedia.org/wiki/Java_ConcurrentMap : Encyclopedia Article

[6]. Web Container (Servlet Container Concept). Available at:

https://en.wikipedia.org/wiki/Web_container : Technical Overview

[7]. ConcurrentHashMap Implementation Reference. Available at:

https://labs.consol.de/java-caches/part-1-4-reference-implementation-with-a-concurrenthashmap/ : Technical Article

[8]. RFC 7231 — HTTP/1.1 Semantics and Methods. Available at:

https://datatracker.ietf.org/doc/html/rfc7231 : Internet Standard Document